Hey ChatGPT, tomorrow I have the interview, and in that interview I want to prepare myself. So I will deliver the technologies. So please generate for me some important topics for revise. So I want to prepare myself for the core Java, advanced Java, collection, Spring Boot, Spring MVC, Spring Boot JPA, Spring Batch, then logical SQL query, Java 8 Stream API question, new latest Java features, design patterns, then microservices related interview things, microservices design pattern, and then Kafka, Docker, and whatever required in today's market for the interview. So that all things. So generate for me for preparation.

Absolutely. Since your interview is tomorrow and you have around 10 years of Java experience, I recommend preparing at a **Senior Java Full Stack Developer / Senior Java Backend Developer level**.

You should not try to learn every technology from scratch tonight. Your objective should be to:

1. Revise important concepts and definitions.
2. Prepare frequently asked technical interview questions.
3. Practise coding and logical SQL queries.
4. Understand real-time production scenarios.
5. Prepare architecture and microservices design questions.
6. Be confident explaining your actual project experience.

I've prepared a **Senior Java Developer Interview – One-Day Revision Checklist (2026)** covering the technologies you mentioned and additional topics expected in today's interviews.

Senior Java Developer – Complete Interview Revision Checklist

Your revision roadmap

16	Senior
Technology areas	Interview level
10 years	1 day
Experience target	Revision timeline

Recommended preparation priority

- **High** Core Java, Collections, Java 8, Spring Boot, Microservices, SQL
- **Medium** JPA, Hibernate, Spring MVC, Kafka, Design Patterns
- **Additional** Spring Batch, Docker, Java latest features, CI/CD

1. Core Java – Must prepare

Very high priority

- OOP concepts: Encapsulation, Inheritance, Polymorphism, Abstraction.
- Abstract class vs Interface.
- Method overloading vs Method overriding.
- Constructor and constructor chaining.
- `this` vs `super`.
- `static`, `final`, `finally`, `finalize`.
- String vs StringBuilder vs StringBuffer.
- String immutability and String Pool.
- `==` vs `equals()`.
- `equals()` and `hashCode()` contract.
- Shallow copy vs Deep copy.
- Immutable class creation.
- Exception handling: Checked vs Unchecked.
- Custom exceptions.
- Try-with-resources.
- Serialization and Deserialization.
- Java Memory Model.
- Stack vs Heap memory.
- Garbage Collection.
- Strong, Weak, Soft and Phantom references.
- JVM, JRE and JDK.
- ClassLoader and class loading process.
- Multithreading and concurrency.
- Thread lifecycle.
- `synchronized` vs `volatile`.
- `wait()` vs `sleep()` vs `notify()`.
- ExecutorService and ThreadPoolExecutor.
- Callable vs Runnable.
- CompletableFuture.
- Deadlock, Race Condition and Thread Safety.

Important scenario questions:

1. How do you create an immutable class?
2. How does Java handle memory management?
3. How would you troubleshoot high CPU usage?
4. How do you identify and resolve deadlocks?
5. How does HashMap work internally?
6. How do you handle exceptions in enterprise applications?

2. Java Collections Framework

Very high priority

- Collection vs Collections.
- List vs Set vs Map.
- ArrayList vs LinkedList.
- HashSet vs LinkedHashSet vs TreeSet.
- HashMap vs Hashtable vs ConcurrentHashMap.
- HashMap internal working.
- HashMap collision handling.
- HashMap equals and hashCode.
- HashMap vs TreeMap.
- Comparable vs Comparator.
- Fail-fast vs Fail-safe iterators.
- Iterator vs ListIterator.
- ConcurrentHashMap internal working.
- How to make collections thread-safe?
- ArrayList internal capacity and resizing.
- CopyOnWriteArrayList.
- BlockingQueue.
- Queue vs Deque.
- Immutable collections.

Must practise: Find duplicates, frequency of elements, sort Map by value, remove duplicates, and find the first non-repeating character.

3. Java 8 Stream API and Functional Programming

Very high priority

- Functional Interface.
- Lambda Expressions.
- Predicate, Function, Consumer, Supplier.
- Stream API.
- Intermediate vs Terminal operations.
- `map()` vs `flatMap()`.
- `filter()`, `sorted()`, `distinct()`.
- `reduce()` vs `collect()`.
- `findFirst()` vs `findAny()`.
- `orElse()` vs `orElseGet()`.
- Optional class.
- Method references.
- Parallel Streams.
- GroupingBy and PartitioningBy.
- Collectors.toMap().
- Stream lazy evaluation.
- Primitive streams: IntStream, LongStream, DoubleStream.

Must practise these coding questions:

1. Find duplicate elements in a list.
2. Find the second-highest number.
3. Find the highest salary employee.
4. Group employees by department.
5. Count employees in each department.
6. Sort employees by salary.
7. Find the first non-repeated character.
8. Find the frequency of each character.
9. Convert List to Map.
10. Find employees whose salary is greater than 50,000.
11. Find the highest-paid employee in each department.
12. Find the second-highest salary in each department.

4. Latest Java Features – Java 11, 17, 21 and beyond

Important

Concentrate on Java 17 and Java 21 because these are important LTS versions.

Version	Important features
Java 8	Lambda, Stream API, Optional
Java 11	HttpClient, var in lambda, String methods
Java 17	Sealed classes, Records, Pattern Matching for instanceof
Java 21	Virtual Threads, Sequenced Collections, Record Patterns, Pattern Matching for switch
Java 25	Newer LTS release; understand its major changes at a high level

Prepare these questions:

- What are Records in Java?
- What are Sealed Classes?
- What are Virtual Threads?
- Platform Threads vs Virtual Threads.
- What is Pattern Matching?
- What are Text Blocks?
- What is the Java Module System?
- Java 8 vs Java 17 vs Java 21.
- How would you migrate a Java 8 application to Java 21?
- What benefits do Virtual Threads provide in high-concurrency applications?

5. Spring Framework and Spring MVC

Very high priority

- What is Spring Framework?
- IoC and Dependency Injection.
- BeanFactory vs ApplicationContext.
- Constructor Injection vs Setter Injection.
- `@Component`, `@Service`, `@Repository`, `@Controller`.
- `@RestController` vs `@Controller`.
- `@Autowired` and `@Qualifier`.
- Bean scopes.
- Spring Bean lifecycle.
- `@Configuration` and `@Bean`.
- Spring MVC architecture.
- DispatcherServlet internal working.
- HandlerMapping and HandlerAdapter.
- Model, View and Controller.
- RequestMapping vs GetMapping vs PostMapping.
- `@RequestParam` vs `@PathVariable` vs `@RequestBody`.
- `@ResponseBody`.
- Global exception handling using `@ControllerAdvice`.
- Interceptors vs Filters.
- Validation using `@Valid` and `@NotNull`.
- REST API status codes.
- CORS.
- Spring Profiles.

Important scenario questions:

1. Explain the complete lifecycle of a Spring Bean.
2. How does Spring Dependency Injection work internally?
3. What happens when a REST API request reaches DispatcherServlet?
4. How do you implement global exception handling?
5. How do you secure REST APIs?
6. How do you handle validation errors?

6. Spring Boot

Very high priority

- Spring Framework vs Spring Boot.
- Spring Boot auto-configuration.
- `@SpringBootApplication`.
- `@EnableAutoConfiguration`.
- Spring Boot starters.
- Embedded Tomcat.
- Application startup process.

- Auto-configuration conditions.
- `@ConditionalOnClass`, `@ConditionalOnMissingBean`.
- `application.properties` vs `application.yml`.
- Profiles and environment-specific configuration.
- ConfigurationProperties.
- Spring Boot Actuator.
- Health checks and metrics.
- Logging and Logback.
- Spring Boot exception handling.
- Spring Boot testing.
- `@SpringBootTest`.
- `@WebMvcTest`.
- `@MockBean` and Mockito.
- Spring Security and JWT.
- Spring Boot 3 and Jakarta namespace changes.

Must know: Explain what happens internally when you start a Spring Boot application.

7. Spring Boot JPA and Hibernate

Very high priority

- JPA vs Hibernate vs Spring Data JPA.
- Entity lifecycle.
- EntityManager.
- Persistence Context.
- First-level cache vs Second-level cache.
- Lazy vs Eager loading.
- N+1 query problem.
- JPQL vs Native SQL.
- Derived query methods.
- `@Query`.
- `@Transactional`.
- Transaction propagation.
- Transaction isolation levels.
- Dirty Checking.
- Entity relationships:
  - OneToOne
  - OneToMany
  - ManyToOne
  - ManyToMany
- Cascade types.
- Orphan Removal.
- FetchType.
- Optimistic vs Pessimistic locking.
- Pagination and Sorting.
- DTO projection.
- Entity vs DTO.
- `save()` vs `saveAndFlush()`.
- Database indexing.
- Auditing.
- Optimizing slow JPA queries.

Important scenario questions:

1. How do you resolve the N+1 problem?
2. Why is LazyInitializationException thrown?
3. How does `@Transactional` work internally?
4. What happens when an exception occurs inside a transaction?
5. How do you prevent duplicate database updates?
6. How do you optimize a slow database query?

8. Spring Batch

Medium priority

- What is Spring Batch?
- Spring Batch architecture.
- Job vs Step.
- JobLauncher.
- JobRepository.
- ItemReader.
- ItemProcessor.
- ItemWriter.
- Chunk-oriented processing.
- Tasklet vs Chunk.
- Skip and Retry.
- JobParameters.
- JobExecution and StepExecution.
- Restartable jobs.
- Job scheduling.
- Multi-threaded steps.
- Partitioning.
- Batch metadata tables.
- Handling millions of records.
- Failed record handling.
- Transaction management in batch processing.

Scenario: How would you process 10 lakh customer records from a database without loading everything into memory?

9. SQL and Database – Logical Queries

Very high priority

Prepare both SQL concepts and hands-on queries.

SQL concepts:

- Primary Key vs Foreign Key.
- Unique vs Not Null.
- Joins: Inner, Left, Right, Full.
- Self Join.
- Subqueries.
- Correlated Subqueries.
- GROUP BY vs HAVING.
- WHERE vs HAVING.
- UNION vs UNION ALL.
- Indexes and composite indexes.
- Clustered vs Non-clustered indexes (database-dependent terminology).

- Views.
- Stored Procedures.
- Functions.
- Triggers.
- ACID properties.
- Normalization.
- Transactions.
- Query execution plan.
- Window Functions.

Must-practise SQL questions:

1. Find the second-highest salary.
2. Find the third-highest salary.
3. Find duplicate records.
4. Delete duplicate records while keeping one.
5. Find employees earning more than their managers.
6. Find the highest salary department-wise.
7. Find the second-highest salary department-wise.
8. Find employees who joined in the last 30 days.
9. Find departments with more than five employees.
10. Find employees with salary greater than the department average.
11. Find the top three salaries in each department.
12. Find customers who never placed an order.
13. Find consecutive duplicate transactions.
14. Calculate running totals using window functions.

10. Microservices Architecture

Very high priority

- Monolithic vs Microservices.
- Microservices architecture.
- Service decomposition.
- API Gateway.
- Service Discovery.
- Eureka Server.
- OpenFeign.
- REST communication.
- Synchronous vs Asynchronous communication.
- Load Balancing.
- Centralized Configuration.
- Distributed Logging.
- Distributed Tracing.
- Correlation ID.
- Inter-service communication.
- Database per Service.
- Distributed transactions.
- Event-driven architecture.
- API versioning.
- Authentication and Authorization.
- JWT.
- Microservices deployment.
- Health checks.
- Resilience and fault tolerance.

Important scenario questions:

1. How do microservices communicate?
2. How do you handle one service being unavailable?
3. How do you maintain data consistency between services?
4. How do you debug an issue across multiple services?
5. How do you secure microservices?
6. How do you handle high traffic?
7. How do you prevent cascading failures?

11. Microservices Design Patterns

Very high priority

Pattern	Purpose
API Gateway	Single entry point
Service Discovery	Find service instances
Circuit Breaker	Prevent repeated failed calls
Retry	Retry temporary failures
Timeout	Avoid waiting indefinitely
Bulkhead	Isolate failures
Saga	Manage distributed transactions
CQRS	Separate read and write models
Event Sourcing	Store events as state history
Strangler	Gradually replace legacy systems
Database per Service	Independent data ownership
Outbox	Reliably publish database changes
Cache-Aside	Improve read performance

Be ready to explain Circuit Breaker, Saga, Retry, Outbox and CQRS with real-world examples.

12. Kafka

High priority

- What is Apache Kafka?
- Kafka architecture.
- Broker, Producer, Consumer.
- Topic and Partition.
- Offset.
- Consumer Group.
- Replication Factor.
- Leader and Follower.
- Kafka message delivery semantics.
- At-most-once vs At-least-once vs Exactly-once.
- Producer acknowledgements.
- Idempotent Producer.
- Consumer Rebalancing.

- Kafka ordering guarantees.
- Partition key.
- Kafka retention.
- Kafka transactions.
- Retry topics.
- Dead Letter Topic (DLT).
- Schema Registry.
- Kafka vs RabbitMQ.
- Spring Kafka integration.
- Kafka monitoring.

Scenario questions:

1. How do you avoid duplicate message processing?
2. What happens when a consumer goes down?
3. How do you handle failed messages?
4. How do you guarantee message ordering?
5. How do you scale Kafka consumers?
6. What happens when a consumer commits an offset?
7. How do you design reliable event-driven communication?

13. Docker and Deployment

Medium priority

- What is Docker?
- Docker vs Virtual Machine.
- Image vs Container.
- Dockerfile.
- Docker Compose.
- Docker volumes.
- Docker networks.
- Container ports.
- Environment variables.
- Multi-stage Docker build.
- Docker Hub.
- Container health checks.
- Container logs.
- Container restart policies.
- Dockerizing Spring Boot applications.
- Kubernetes basics: Pod, Deployment, Service, ConfigMap, Secret.
- CI/CD pipeline.
- Jenkins build and deployment.

Scenario: How would you deploy a Spring Boot microservice with MySQL and Kafka using Docker Compose?

14. Design Patterns and SOLID Principles

High priority

Creational patterns

- Singleton
- Factory
- Abstract Factory
- Builder
- Prototype

Structural patterns

- Adapter
- Decorator
- Proxy
- Facade
- Composite

Behavioral patterns

- Strategy
- Observer
- Template Method
- Chain of Responsibility
- Command
- State

SOLID principles

- Single Responsibility
- Open/Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

Important questions:

- Where have you used Factory Pattern?
- How does Spring use Singleton Pattern?
- How does Spring AOP use Proxy Pattern?
- How would you implement Strategy Pattern?
- Explain SOLID with a real project example.

15. Unit Testing and Code Quality

Don't skip this section for a senior developer interview.

- JUnit 5.
- Mockito.
- `@Mock` vs `@InjectMocks`.
- Mock vs Spy.
- `when()` vs `doReturn()`.
- `verify()`.
- ArgumentCaptor.
- Unit testing vs Integration testing.
- Testing REST controllers.
- MockMvc.
- Test coverage.
- SonarQube.
- Code quality.
- Test cases for exception scenarios.

16. Additional topics expected in today's market

- Spring Security and JWT authentication.
- OAuth 2.0 basics.

- Redis caching.
- API performance optimization.
- Rate limiting.
- Elasticsearch basics.
- Kubernetes fundamentals.
- Cloud basics: AWS/Azure.
- Git branching and merge strategies.
- Jenkins CI/CD.
- Application monitoring.
- OpenTelemetry and distributed tracing.
- Secure coding and OWASP basics.
- AI-assisted development tools and coding copilots.

You don't need deep knowledge of all these additional topics. Know their purpose and where they fit into a real application.

17. Your one-day preparation timetable

Don't give equal time to all technologies. Follow this sequence.

Time	Preparation
7:00–8:00 AM	Core Java + JVM
8:00–9:00 AM	Collections + Multithreading
9:00–10:00 AM	Java 8 Streams + Coding
10:00–11:00 AM	Spring Framework + Spring MVC
11:00–12:00 PM	Spring Boot
12:00–1:00 PM	JPA + Hibernate
1:00–2:00 PM	Break + SQL queries
2:00–3:00 PM	Microservices architecture
3:00–4:00 PM	Microservices design patterns
4:00–5:00 PM	Kafka
5:00–5:45 PM	Spring Batch
5:45–6:30 PM	Docker + Kubernetes basics
6:30–7:30 PM	Coding + SQL practice
7:30–8:30 PM	Design patterns + JUnit + Mockito
8:30–9:30 PM	Project explanation + mock interview
After 9:30 PM	Light revision and adequate sleep

Adjust the timings based on your actual interview schedule. If you have less time, prioritize the high-priority sections.

18. Prepare your real project explanation

This is especially important because you have worked on the HSBC eShare Admin application.

Prepare these questions:

1. Explain your current project architecture.
2. What is the business purpose of eShare Admin?
3. What is your role and responsibility?
4. Explain your Spring Boot application architecture.
5. How do you interact with Oracle DB?
6. How do you implement REST APIs?
7. How do you integrate third-party APIs?
8. How do you manage user permissions and access rules?
9. How do you handle exceptions?
10. How do you troubleshoot production issues?
11. How do you perform code reviews?
12. How do you deploy applications using Jenkins?
13. How do you handle production incidents?
14. What performance improvements have you implemented?
15. Explain one challenging production issue you resolved.

Prepare one real example for each of these areas: coding, performance, production support, database, and debugging.

19. Senior-level scenario-based questions

These questions help interviewers understand whether you have practical experience rather than only theoretical knowledge.

Scenario	What interviewer expects
API response takes 10 seconds	Performance troubleshooting
Application CPU reaches 100%	Thread dump, profiling
Database connection pool exhausted	Connection pool investigation
Memory usage continuously increases	Heap dump, memory leak
Kafka consumer processes duplicate messages	Idempotency
One microservice is down	Circuit breaker, fallback
Database query is slow	Execution plan, indexing
Application receives 10,000 requests/sec	Scaling, caching, load balancing
Two users update the same record	Optimistic locking
Batch job fails halfway	Restartability, checkpoints
Deployment causes application failure	Rollback, health checks
API receives invalid data	Validation, exception handling

20. Final interview preparation checklist

Your revision progress 0/23 completed

- [ ] Explain OOP concepts with examples
- [ ] Explain HashMap internal working
- [ ] Explain ConcurrentHashMap
- [ ] Revise equals() and hashCode()
- [ ] Revise multithreading and ExecutorService
- [ ] Practise 10 Java 8 Stream programs
- [ ] Revise Java 17 and 21 features

- [ ] Explain Spring Bean lifecycle
- [ ] Explain DispatcherServlet flow
- [ ] Explain Spring Boot auto-configuration
- [ ] Explain @Transactional
- [ ] Explain JPA N+1 problem
- [ ] Practise 10 SQL queries
- [ ] Explain microservices architecture
- [ ] Explain Circuit Breaker and Saga
- [ ] Explain Kafka consumer groups and offsets
- [ ] Explain retries and DLT
- [ ] Explain Dockerfile and Docker Compose
- [ ] Revise SOLID and design patterns
- [ ] Revise JUnit and Mockito
- [ ] Prepare HSBC project explanation
- [ ] Prepare one challenging production issue
- [ ] Practise tell me about yourself

21. My recommendation for tomorrow

At your experience level, interviewers will expect more than definitions.

For every important topic, prepare answers in this format:

1. **What is it?** – Definition in simple English.
2. **How does it work?** – Internal working.
3. **Why do we use it?** – Business purpose.
4. **Real-time example** – Where you used it.
5. **Interview scenario** – How you would solve a problem.

For example, don't just say:

"HashMap stores data in key-value pairs."

Explain hashing, bucket calculation, collision handling, equals/hashCode, resizing and concurrency considerations.

Similarly, don't just define microservices. Explain service communication, failure handling, database consistency, monitoring and deployment.

One important thing: Don't spend tonight trying to master Kubernetes, advanced AWS or every new Java feature. Strengthen Java, Spring Boot, SQL, JPA and Microservices first.

Let's practise your interview

I recommend that we now conduct a senior-level mock interview with questions one by one. You answer in your own words, and I'll evaluate your technical correctness, identify missing points and improve your answer into interview-ready English.

I can also prepare a fresh rapid-revision set for tomorrow morning.

Send tomorrow's Java revision questions
Tap to set up

Your immediate next step: Start with Core Java. Be ready to explain HashMap internal working, ConcurrentHashMap, multithreading, Java 8 Streams and Spring Boot architecture without looking at notes.